TECHNICAL SUPPORT REPORT

STM32N6570-DK boots to silence at the FSBL→Application handoff

FSBL hangs immediately before calling `BOOT_Application()` when run standalone in Flash Boot — but the identical code and binaries work correctly under a debugger in DEV boot. Prepared for a support call with ST.

Board STM32N6570-DK Rev Z **Device ID** 0x486 **ST-LINK FW** V3J17M11 **CubeIDE** 2.2.0

CubeProgrammer v2.23.0 **Signing Tool** v2.23.0 (header v2.3)

---

01 What we're trying to do

Our firmware uses ST's standard two-stage XIP boot pattern for the STM32N6:

- An **FSBL** (First Stage Boot Loader) runs first from internal flash at `0x34180400` (the nonsecure alias). It brings up USART1, XSPI1/XSPI2, and ST's `STM32_ExtMem_Manager` middleware.
- The FSBL then memory-maps an **external XSPI NOR flash** and jumps to an **Application** image that executes in place (XIP) directly from that external flash at `0x70100000`.
- The Application image is signed with ST's Signing Tool, header v2.3, binary type `0x10` (`fsbl` type value, empty-key header, load address `0x70100000`).

This is exactly the pattern in ST's own `STM32_ExtMem_Manager` boot-XIP example and the "How to debug STM32N6 using STM32CubeIDE" community article — we haven't deviated from the reference architecture.

02 The symptom

> REPRODUCIBILITY

100% reproducible. Confirmed across many power cycles and physical reset-button presses, using freshly rebuilt and freshly reflashed binaries each time. Not a one-off glitch.

In **Flash Boot mode** (BOOT0 and BOOT1 both held low — the board's normal standalone boot, no debugger attached), the FSBL prints its boot trace over USART1 (115200 8N1) right up to one specific line, then goes completely silent forever. No further UART output, no fault, no watchdog reset — the board just sits there.

```
✓ printed FSBL: UART alive
```

```
✓ printed FSBL: XSPI1 init done
```

```
✓ printed FSBL: XSPI2 init done
```

```
✓ printed FSBL: EXTMEM_MANAGER init done
```

```
✓ printed FSBL: init done, calling BOOT_Application
```

```
✗ never appears BOOT: calling MapMemory  (first line inside BOOT_Application())
```

WHY THIS IS A STRANGE PLACE TO HANG

The line that *does* print and the line that *never* prints use the **identical** transmission mechanism: a direct, blocking `HAL_UART_Transmit(&huart1, ...)` call, same UART handle, same peripheral, same call pattern. There is no libc/newlib stdio involved on either side. The only thing between the last successful print and the first missing one is a plain C function call into `BOOT_Application()`.

03 The critical clue: it works under a debugger

CONFIRMED WORKING

The **exact same compiled code**, when run in **DEV boot** (BOOT1 high / BOOT0 low) with STM32CubeIDE attached via ST-LINK, boots correctly and completely:

- `MapMemory()` is reached and returns `0` (success)
- The external NOR flash is successfully placed into memory-mapped mode

- `JumpToApplication()` correctly reads SP/PC from the Application's vector table at the mapped address
- Execution jumps into the Application, which begins running normally

So the hang is specific to the **standalone Flash Boot path**. Something differs in hardware/execution state between ST's boot-ROM DEV-boot path (which halts and waits for a debugger, opening debug access automatically) and its Flash-Boot path (which runs straight through, unattended) — a difference that is breaking a plain function call/UART transmit sequence that otherwise works identically on both paths.

04 What we've already investigated and ruled out

Three real bugs were found and fixed along the way. **None of them resolved the hang** — it persisted, identically, after each fix, rebuild, and reflash.

FIXED — UNRELATED TO THIS HANG

1. Debug print after VTOR/MSP switch in JumpToApplication()

A separate, distinct crash: a debug print sat *after* `SCB->VTOR` and the stack pointer were already switched to the Application's own (not-yet-initialized) exception context. Any UART hiccup during that print would vector through the Application's exception table instead of the FSBL's. Confirmed live under the debugger: a hard fault landed in unmapped code with no debug information, inside the print's call stack. **Fixed** by removing that specific print. This is downstream of — and unrelated to — the hang we're reporting here, since execution never even reaches this function anymore.

FIXED, HANG PERSISTED IDENTICALLY

2. Suspected newlib/snprintf reentrancy

Our leading hypothesis was that `snprintf()`-based debug prints (routing through newlib's reentrant `_svfprintf_r`/`__ssputs_r`) aren't safe this early in a bare-metal, non-RTOS boot context. We eliminated **every** `snprintf` call from the FSBL's boot/XIP path entirely, replacing them with hand-written direct `HAL_UART_Transmit` helpers (string length by hand, hex/decimal formatting by hand — no libc stdio touched at all). Verified via source grep that zero live `snprintf`/`DBG_PRINT` calls remain. Rebuilt cleanly, reflashed a fresh binary.

Result: the hang reproduced at the exact same point, byte-for-byte identical symptom. This was a real code-quality fix worth keeping, but it is not the cause of this hang.

FIXED — UNEXERCISED (NEVER REACHED)

3. Buffer overrun in SAL_XSPI_EnableMapMode() debug log

A `snprintf` call into a 128-byte buffer whose formatted output could exceed 128 bytes, with the unclamped return value passed straight to `HAL_UART_Transmit` — reading past the buffer. This matched garbled/truncated log output we'd separately observed on a different boot attempt. Fixed with a length clamp. Downstream of the current hang point, so not yet exercised on real hardware.

05 Environment details

ITEM	VALUE
Board	STM32N6570-DK, Rev Z
Device ID / BL Version	0x486 / --
ST-LINK	FW V3J17M11
Toolchain	STM32CubeIDE 2.2.0 (bundled arm-none-eabi-gcc)
Flash tool	STM32CubeProgrammer v2.23.0
Signing	STM32_SigningTool_CLI v2.23.0 — header v2.3, `-t fsbl -of 0x0 -nk -align`, load address `0x70100000`, dumped binary type `0x10`
FSBL link address	`0x34180400` (internal flash, nonsecure alias)
Application link address	`0x70100000` (external XSPI NOR, memory-mapped/XIP)
Build flags	`-mcpu=cortex-m55 -mcmse -O0 -mfpu=fpv5-d16 -mfloat-abi=hard -mthumb --specs=nano.specs`

06 Relevant source

Core/Src/main.c — the last successful print and the call that never returns

```
MX_EXTMEM_MANAGER_Init();
FSBL_UART_PUTS("FSBL: EXTMEM_MANAGER init done\r\n");  // ✓ prints fine

FSBL_UART_PUTS("FSBL: init done, calling BOOT_Application\r\n");  // ✓ prints fine — last output ever

/* Launch the application */
if (BOOT_OK != BOOT_Application())  // ← hangs somewhere in/around this call
{
  FSBL_UART_PUTS("FSBL: BOOT_Application FAILED\r\n");  // never reached
  Error_Handler();
}
```

Middlewares/ST/STM32_ExtMem_Manager/boot/stm32_boot_xip.c — entry point, first line never executes

```
BOOTStatus_TypeDef BOOT_Application(void)
{
  BOOTStatus_TypeDef retr;

  /* This is the very first statement in the function.
     It uses the exact same HAL_UART_Transmit() call pattern
     as the FSBL_UART_PUTS() line above that DID print. */
  dbg_puts("BOOT: calling MapMemory\r\n");  // ← never observed on the wire
  retr = MapMemory();
  ...
}
```

where `dbg_puts` is a hand-written helper with no libc stdio dependency:

```
static void dbg_puts(const char *s)
{
  uint16_t len = 0u;
  while (s[len] != '\0') { len++; }
  HAL_UART_Transmit(&huart1, (uint8_t *)s, len, HAL_MAX_DELAY);
}
```

07 New evidence: the CPU's own debug access port goes dark while hung

We connected STM32CubeProgrammer directly to the board via SWD **Hot Plug** mode (attaches without issuing any reset, so whatever state the board is in is

preserved exactly as-is), and ran the same check under three different conditions to build a controlled comparison rather than a single data point.

CONDITION	ACCESS PORT	CONNECT / ID READ	MEMORY READ (`0XE000ED28` ETC.)
DEV boot (BOOT1 hi / BOOT0 lo), core idling in boot ROM	1 *(auto-negotiated)*	Succeeds — Device 0x486, Rev B, core ST32N657 identified	**Succeeds.** CFSR/HFSR/DFSR all read `0x00000000` — no fault has occurred, as expected since our code hasn't run yet.
Flash Boot (both low), hung at the reported point	0 *(default)*	Partially succeeds — Device ID 0x486 resolves at the debug-port level	**Fails.** Every read (three different addresses tried) immediately fails and drops the connection: "Warning: Connection to device 0x486 is lost."
Flash Boot (both low), hung at the reported point	1 *(the same AP confirmed working under DEV boot)*	**Fails outright** — "Error: Unable to get core ID", "Error: Cannot identify the device"	N/A — couldn't get far enough to try

WHAT THIS SHOWS

This isn't "we probed some address the tool doesn't like" — it's the same debugger, the same cabling, the same ST-LINK, reading the same **ARM-standard** register (`0xE000ED28`, not something STM32N6-specific), through the exact Access Port that is independently confirmed to work correctly (DEV boot row above). The only variable that changed is which boot path the chip took. In Flash Boot's hung state, the CPU's own core access port (AP1) is **completely unreachable** — not degraded, not slow, entirely unresponsive to identification, let alone memory access. AP0 gets slightly further (a debug-port-level ID resolves) but still cannot complete any actual bus transaction.

This points to a hardware/system-level condition specific to the Flash Boot path — something is leaving the core (or the bus fabric immediately around it) in a state where even independent, CPU-uninvolved debug hardware can't reach it. That's consistent with (though not proof of) a blocked/illegal bus transaction that never resolves, per the RIF/IAC theory referenced in the questions below, but the AP1-totally-dark result is the strongest single piece of evidence we have that this is not just an application code bug.

08 Questions for ST

Q1 What differs in hardware/peripheral state at this point in boot between the boot-ROM's **DEV boot** path (halt, open debug access, wait for debugger) and its **Flash Boot** path (run straight

through unattended) — specifically anything that could silently stall a plain function call / blocking UART transmit, with no fault and no watchdog reset?

Q2 Is there a known interaction between **IWDG/WWDG** state, **MPU/cache configuration**, or **interrupt/exception table setup** that behaves differently across these two boot paths on the STM32N6 specifically?

Q3 Are there any known XSPI/EXTMEM peripheral **clock-gating** differences between DEV boot and Flash Boot that could leave a peripheral un-clocked (or in a different power/domain state) only in the standalone path?

Q4 Is there a documented minimum/required step between "last FSBL init" and "call into the boot/XIP module" that our reference code might be missing — something the debugger's connect sequence happens to paper over (e.g. an implicit reset or clock reconfiguration it performs) but standalone Flash Boot does not?

Q5 Via Hot-Plug SWD attach (no reset issued), we confirmed that Access Port 1 (verified working and able to identify/read memory correctly under DEV boot with the identical debugger and cabling) becomes **completely unreachable** — can't even identify the core — while the board is hung in Flash Boot. Does this match a known STM32N6 failure mode where a stalled/blocked bus transaction (e.g. an unresolved illegal access) leaves the core's own debug access port unresponsive? What conditions cause a transaction to stall rather than fault cleanly, and is there any way to recover from or further diagnose this state (e.g. via JTAG boundary scan, or reading IAC/RIFSC status through a different access path that doesn't depend on AP1)?
